
Natural Language Processing basics

— Intro to AI - Autumn 2025 Class —

Eleni Metheniti - eleni.metheniti@univ-tlse3.fr

Table of contents

1. [What is NLP?](#)
2. [Fundamentals of linguistics](#)
3. [Natural Language Preprocessing](#)
4. [Getting started in NLP](#)
 - 4.1. [Our tools for NLP](#)
 - 4.2. [Representing data](#)
 - 4.3. [Processing data](#)
5. [Final thoughts](#)

1. What is NLP?

NLP = Natural Language Processing

- Natural Language
 - Any language that occurs naturally in a human community by process of use, repetition, and change
 - **Oral*** and **written** production

NLP = Natural Language Processing

- Natural Language
 - Any language that occurs naturally in a human community by process of use, repetition, and change
 - **Oral*** and **written** production
- Processing... but with what?
 - **Linguistics:** studying natural language
 - **Mathematics:** formal logic, statistics & algorithms
 - **Computer Science:** implementing algorithms and machine learning models
 - **Physics:** speech processing as wavelengths

NLP in a nutshell

The application of computational techniques for the analysis and synthesis of natural language and speech

- **Text and speech processing:** speech recognition, text-to-speech, etc.
- **Morphological analysis:** lemmatization, part-of-speech tagging, etc.

NLP in a nutshell

The application of computational techniques for the analysis and synthesis of natural language and speech

- **Text and speech processing:** speech recognition, text-to-speech, etc.
- **Morphological analysis:** lemmatization, part-of-speech tagging, etc.
- **Syntactic analysis:** sentence boundary disambiguation, parsing, etc.
- **Lexical semantics:** named entity recognition, sentiment analysis, etc.
- **Relational semantics:** relationship extraction, semantic parsing, etc.
- **Discourse analysis:** coreference resolution, topic segmentation, etc.

NLP in a nutshell

The application of computational techniques for the analysis and synthesis of natural language and speech

- **Text and speech processing:** speech recognition, text-to-speech, etc.
- **Morphological analysis:** lemmatization, part-of-speech tagging, etc.
- **Syntactic analysis:** sentence boundary disambiguation, parsing, etc.
- **Lexical semantics:** named entity recognition, sentiment analysis, etc.
- **Relational semantics:** relationship extraction, semantic parsing, etc.
- **Discourse analysis:** coreference resolution, topic segmentation, etc.
- **Higher level NLP applications:** summarization, machine translation, chatbots, etc.

NLP in a nutshell

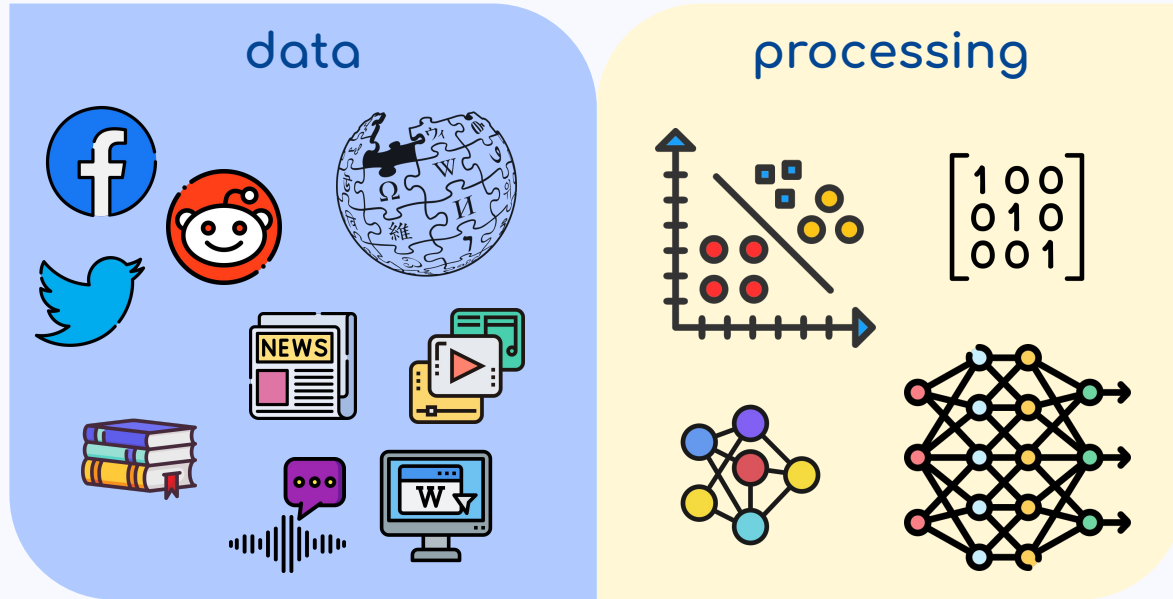
The application of **computational techniques** for the analysis and synthesis of natural language and speech

- up to 1980s: **Rule-based methods**, e.g. regular expressions, hand-written rules
 - created by experts, precise but difficult to scale
- 1990s: **Statistical methods**, e.g. linear functions
 - treating language as numerical data, deducing/learning patterns
- 2010s: **Neural networks**, a combination of linear and nonlinear functions
 - state-of-the-art performance, but harder to interpret
- 2020s: **Multi-layer, multi-head transformer neural networks**

NLP in a nutshell



NLP in a nutshell

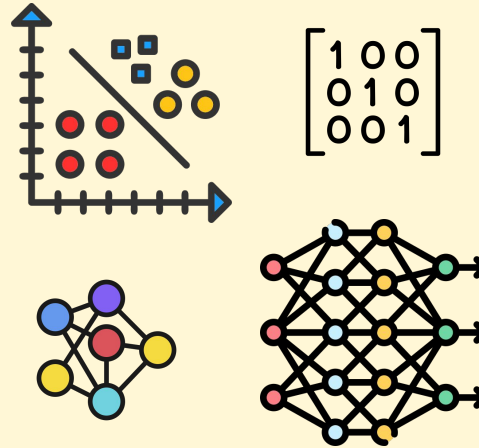


NLP in a nutshell

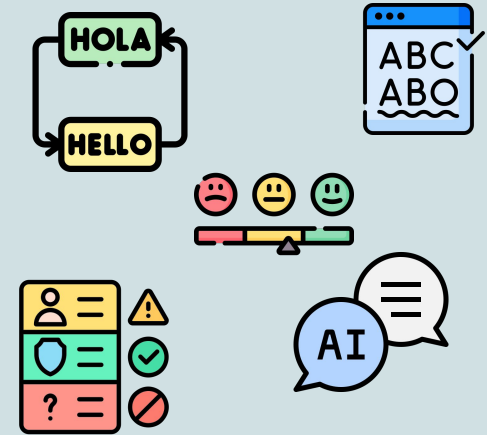
data



processing



applications



2. Fundamentals of linguistics

What is linguistics?

The study of spoken & written natural language w.r.t. form, meaning, and context

What is linguistics?

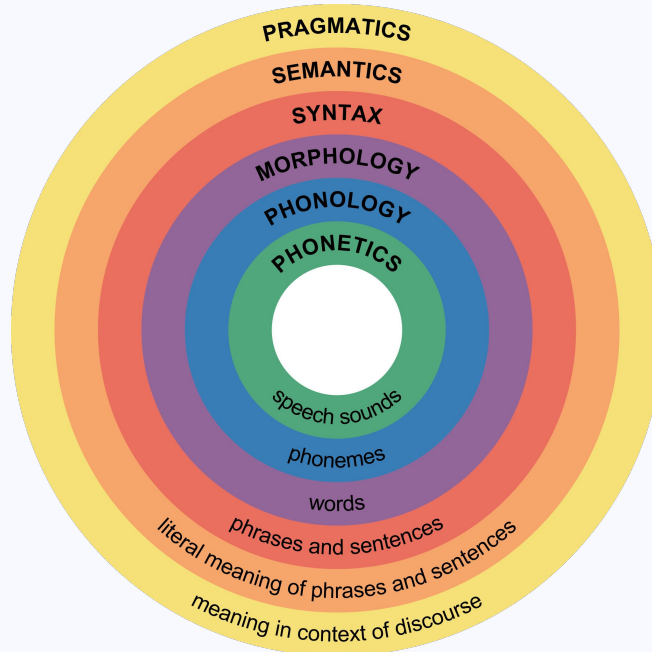
The study of spoken & written natural language w.r.t. form, meaning, and context

Phonetics:

Physical aspects of speech sounds

Phonology:

Linguistic sounds of a particular language



What is linguistics?

The study of spoken & written natural language w.r.t. form, meaning, and context

Phonetics:

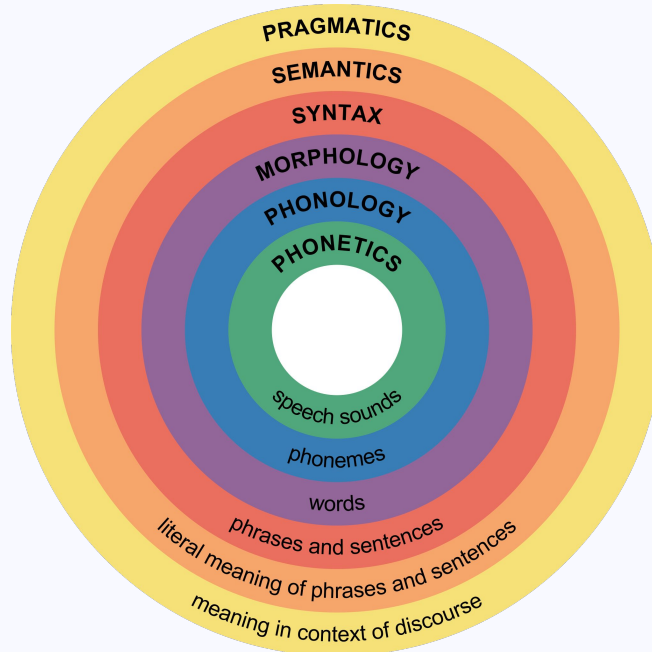
Physical aspects of speech sounds

Phonology:

Linguistic sounds of a particular language

Morphology:

Senseful components of words and word forms



What is linguistics?

The study of spoken & written natural language w.r.t. form, meaning, and context

Phonetics:

Physical aspects of speech sounds

Phonology:

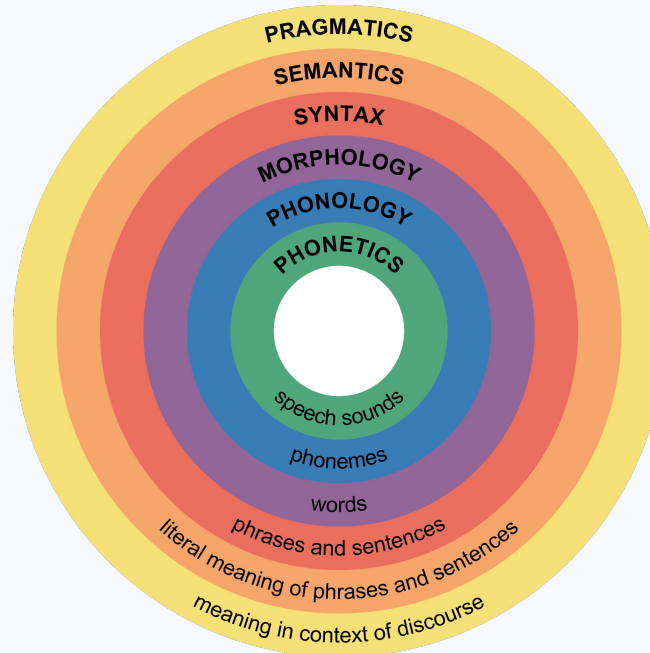
Linguistic sounds of a particular language

Morphology:

Senseful components of words and word forms

Syntax:

Structural relationships between words, usually in a sentence



What is linguistics?

The study of spoken & written natural language w.r.t. form, meaning, and context

Phonetics:

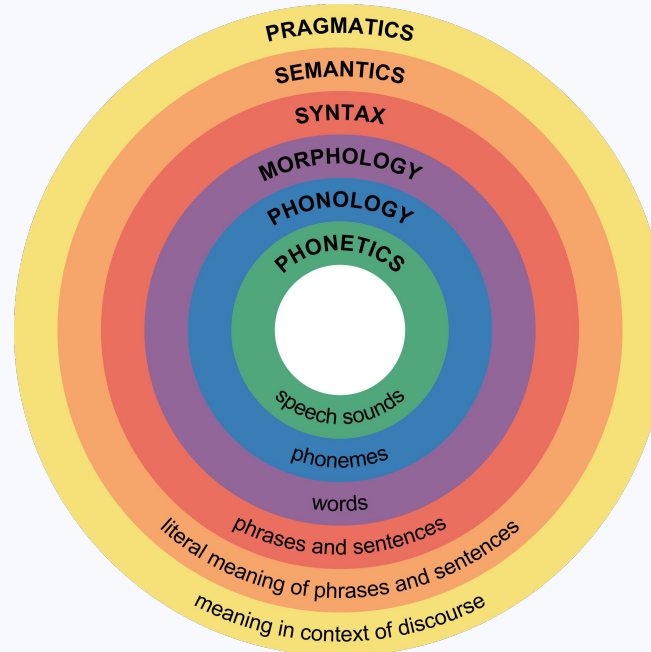
Physical aspects of speech sounds

Phonology:

Linguistic sounds of a particular language

Morphology:

Senseful components of words and word forms



Syntax:

Structural relationships between words, usually in a sentence

Semantics:

Meaning of single words and compositions of words

Pragmatics:

How language is used to accomplish goals

Linguistic building blocks

- **Phoneme:** Smallest unit of spoken language (usually one speech sound)

On		a		laissé				la		fenêtre								ouverte.							
on		a		l	ai	ss	é		l	a		f	e	n	ê	t	r	e		ou	v	e	r	t	e
õ		a		l	e	s	e		l	a		f	ə	n	ɛː	t	ʁ	ə		u	v	ɛ	ʁ	t	ə

on	a	lai	ssé	la	fe	ne	tre	ou	ver	te
õ 'na		le	'se	laf		'nɛʁ		u	'vɛʁt	

https://en.wikipedia.org/wiki/French_phonology

Linguistic building blocks

- **Morpheme:** Smallest unit with a meaning or grammatical function in both spoken and written language

ajouter (V)	
ajout	er
<i>stem</i>	<i>suffix</i>
<i>meaning</i>	<i>function</i>

ajoutent (V)	
ajout	ent
<i>stem</i>	<i>suffix</i>
<i>meaning</i>	<i>function</i>

lemma: ajouter (V)

stem: ajout-

Linguistic building blocks

- **Morpheme:** Smallest unit with a meaning or grammatical function in both spoken and written language

photocopie (N)	
photo	copie
<i>stem</i>	<i>stem</i>
<i>meaning</i>	<i>meaning</i>

photocopies (N)		
photo	copie	s
<i>stem</i>	<i>stem</i>	<i>suffix</i>
<i>meaning</i>	<i>meaning</i>	<i>function</i>

photocopies (V)	
photocopi	es
<i>stem</i>	<i>suffix</i>
<i>meaning</i>	<i>function</i>

lemma: photocopie (N)
photocopier (V)

stem: photocopi-

Linguistic building blocks

- **Morpheme:** Smallest unit with a meaning or grammatical function in both spoken and written language

anticonstitutionnellement (ADV)				
anti	constitu	tion(n)	elle	ment
<i>prefix</i>	<i>stem</i>	<i>suffix</i>	<i>suffix</i>	<i>suffix</i>
<i>function</i>	<i>meaning</i>	<i>meaning</i>	<i>function</i>	<i>function</i>

lemma: anticonstitutionnellement (ADV)

stem: constitu-

Linguistic building blocks

- **Word?**

- **Simple terms:** the string of letters between two spaces or punctuation?
but... *pin's*, *sac à dos*, *Tour Eiffel* are one word each!
- **Phonetics/Phonology:** the cluster of sounds between two pauses?
but... we often pronounce *je suis* as *chui* !
- **Morphology:** a cluster of morphemes with a distinct meaning and distinct function?
but... if we split *langue-de-chat* 🍞 , it will become *langue de chat* ! 🐱

Linguistic building blocks

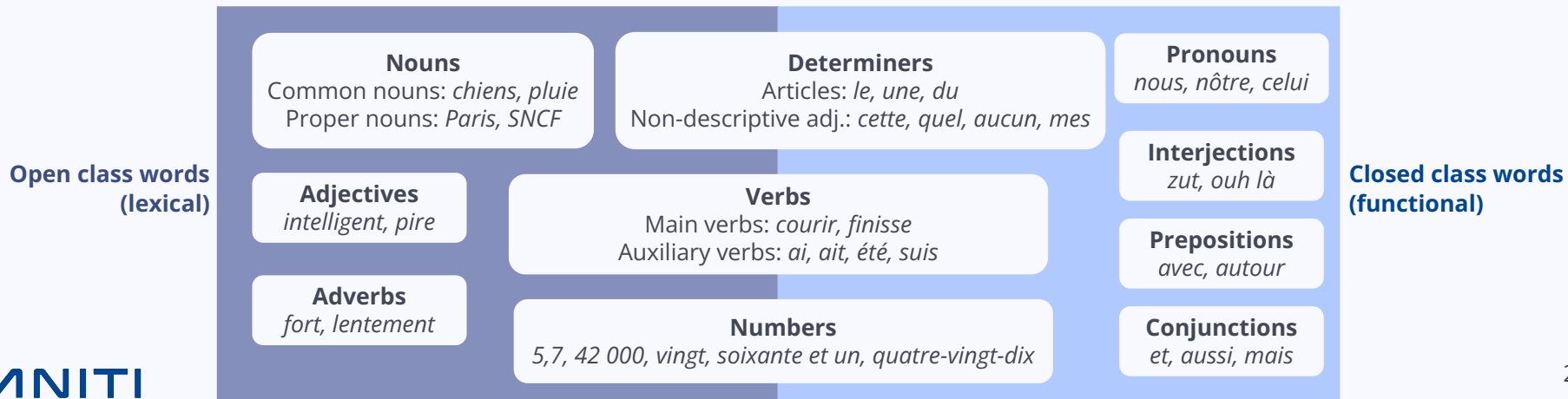
- **Word**

- The smallest unit of language that is to be uttered in isolation
- Has either a lexical function (e.g. verbs, nouns) or a grammatical function (e.g. articles)
- Every word is composed of one or more morpheme(s): (prefix) - **stem** - (suffix)

Linguistic building blocks

- **Word**

- The smallest unit of language that is to be uttered in isolation
- Has either a lexical function (e.g. verbs, nouns) or a grammatical function (e.g. articles)
- Every word is composed of one or more morpheme(s): (prefix) - **stem** - (suffix)



Linguistic building blocks

- **Sentence?**

- **Simple terms:** a string of words starting with a capital letter and ending in punctuation?
but... where does *"Jules César a été assassiné en 44 av. J.-C. en plein sénat."* start/end?
- **Syntax:** a unit consisting of a subject and predicate?
but... *"Métro, boulot, dodo."* doesn't have these elements, but is a sentence!
- **Semantics:** a cluster of words with a complete meaning?
but... *"Yes!"* is a sentence, with complete meaning in the context of a discussion

Linguistic building blocks

- **Sentence:** a set of words that is
 - complete in itself
 - (typically) contains a subject and predicate
 - (typically) conveys a statement, question, exclamation, or command
 - (typically) consists of a main clause and sometimes one or more subordinate clauses
- **Phases → Sentences → Clause**

Clause:	Je pense que ton chien aime Marie.				
Sentence:	je pense		(que) ton chien aime Marie		
Phrase:	je	pense	ton chien	aime	Marie

Linguistic building blocks



And yet... we're blocked

Even when sentences are simple... semantics are not!

- **Lexical ambiguity:**

Je mange un avocat. ⇒   or   ?

- **Syntactic ambiguity:**

L'artiste peint la nuit. ⇒  or  ?

- **Grammaticality vs. Plausibility:**

La banane court un marathon. ⇒   ?

And yet... we're blocked

Even when sentences are simple... semantics are not!

- **Coreference:**

Sylvain a caressé son chien et il lui a donné une friandise. ⇒  or  ?

- **Pragmatic ambiguity:**

Sylvain a vu un homme avec un télescope. ⇒ Who has the telescope?

- **Garden path sentences:**

*Sylvain a vu un homme avec un télescope, qu'il a ensuite ramené chez lui
pour lui demander conseil.*

And yet... we're blocked

Even when sentences are simple... semantics are not!

- **Sarcasm:**

Oh mais j'adore faire la queue à la banque! ⇒ Wait, really?

- **Negation and contrast:**

Mes nouvelles baskets sont moches mais tellement confortables. ⇒ ★ or ★★★★★?

And yet... we're blocked

Language in grammar books $\neq$ Language IRL!

- Slang, neologisms, abbreviations, etc.
- Spelling errors, neologisms, creativity, etc.
- Inference from world knowledge



bewbin Follow

in World War 1 around 8 million horses died but in World War 2 it was under a million which can only mean horses started to evolve bullet resistance

<https://bewbin.tumblr.com/>

Wsh tu viens à ma soirée?

Kel heur?

Jsp frr 21h



<https://x.com/NathlnReims/status/659716375099871232>

3. Natural Language Preprocessing

Processing text:

- A word
 - Tokenization
 - Lemmatization/Stemming

Processing text:

- A word
 - Tokenization
 - ~~Lemmatization/Stemming~~
- A sentence
 - Lower-casing
 - Segmentation
 - Lemmatization/Stemming
 - Part-of-speech tagging
 - Stop word removal
 - Syntactic analysis

Processing text:

- A word
 - Tokenization
 - ~~Lemmatization/Stemming~~
- A sentence
 - Lower-casing
 - Segmentation
 - Lemmatization/Stemming
 - Part-of-speech tagging
 - Stop word removal
 - Syntactic analysis
- A paragraph
 - Syntactic analysis
 - Word disambiguation
 - Named Entity Recognition
 - Coreference resolution
 - Discourse parsing

Tokenization

Separate a word to: morphemes for humans, **tokens** for machines!

- **Character tokens:**

J'écoute les oiseaux.																		
j	'	é	c	o	u	t	e	l	e	s	o	i	s	e	a	u	x	.

- **Subword tokens:**

J'écoute les oiseaux.							
j'	écout	e	les	ois	eau	x	.

We determine these subtokens based on frequency or with algorithms (e.g. [BPE](#))

- **Word tokens:**

J'écoute les oiseaux.					
j'	écoute	le	s	oiseaux	.

Tokenization

Vocabulary (NLP):

Set of all tokens in the data we are using

Separate a word to: morphemes for humans, **tokens** for machines!

*small vocabulary
many tokens/sentence*

- **Character tokens:**

J'écoute les oiseaux.																		
j	'	é	c	o	u	t	e	l	e	s	o	i	s	e	a	u	x	.

- **Subword tokens:**

J'écoute les oiseaux.							
j'	écout	e	les	ois	eau	x	.

- **Word tokens:**

J'écoute les oiseaux.					
j'	écoute	le	s	oiseaux	.

*large vocabulary
few tokens/sentence*

Lower-casing, Segmentation

- Removing capitalization, splitting a sentence in words

J'écoute les oiseaux.				
j'	écoute	les	oiseaux	.

Part-of-speech tagging

- Labeling words with their part-of-speech type, based on the current sentence

J'écoute les oiseaux.				
j'	écoute	les	oiseaux	.
PRON	VERB	DET	NOUN	PUNCT

Lemmatization/Stemming

- Turning the word to its lemma and stem

J'écoute les oiseaux.				
j'	écoute	les	oiseaux	.
PRON	VERB	DET	NOUN	PUNCT

<i>je</i>	<i>écouter</i>	<i>le</i>	<i>oiseau</i>	.
<i>je</i>	<i>écout</i>	<i>le</i>	<i>oiseau</i>	.

Why not at word level?

Meaning in sentence!

Stop word removal

- Removing high-frequency words that do not contribute to sentence's unique meaning

J'écoute les oiseaux.				
j'	écoute	les	oiseaux	.
PRON	VERB	DET	NOUN	PUNCT

<i>je</i>	<i>écouter</i>	<i>le</i>	<i>oiseau</i>	.
<i>je</i>	<i>écout</i>	<i>le</i>	<i>oiseau</i>	.

<i>j'</i>	<i>écoute</i>	<i>les</i>	<i>oiseaux</i>	.
-----------	---------------	-----------------------	----------------	---

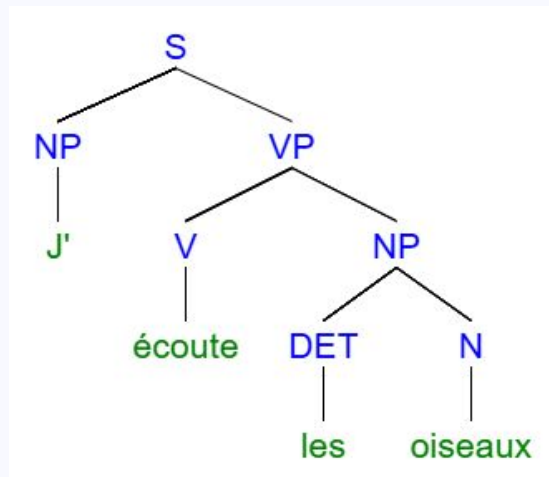
How do we find these stop words?

With statistics...

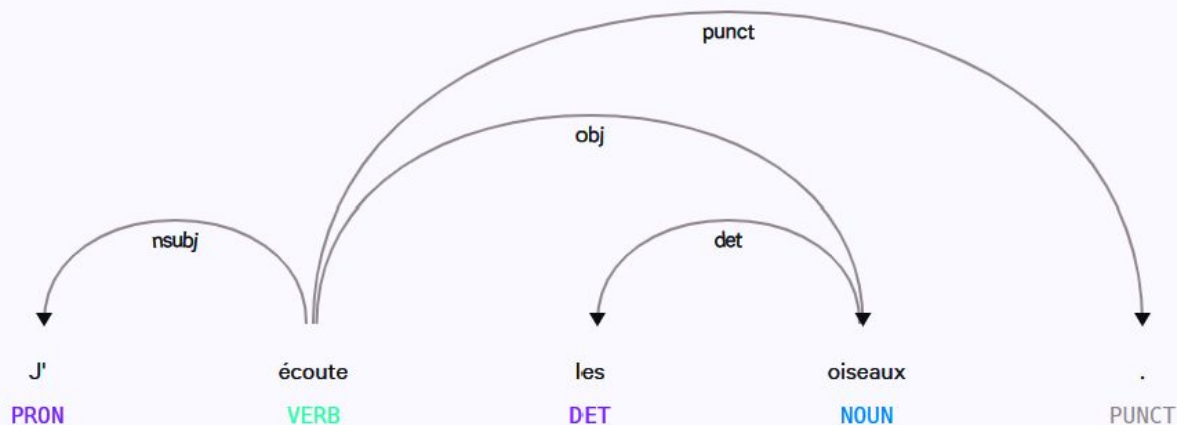
...but we can also use premade [tools](#) and [lists](#)

Syntactic parsing

- Syntactic analysis of a sentence
- **Human-readable way:** constituency graphs or dependency graphs



<https://mshang.ca/syntree/>



<https://demos.explosion.ai/displacy>

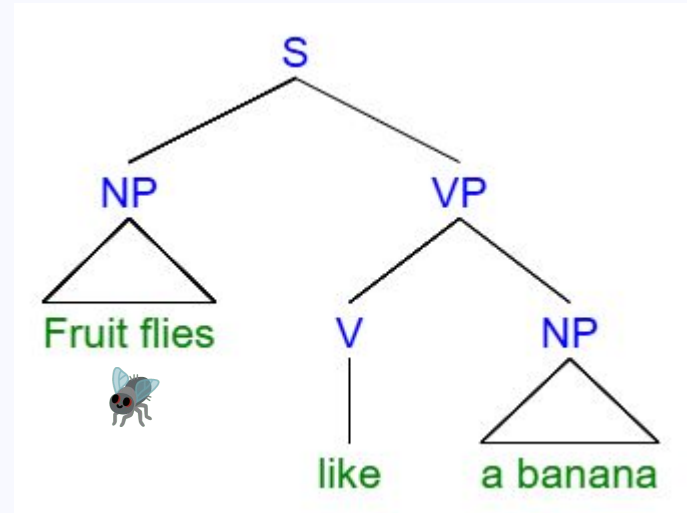
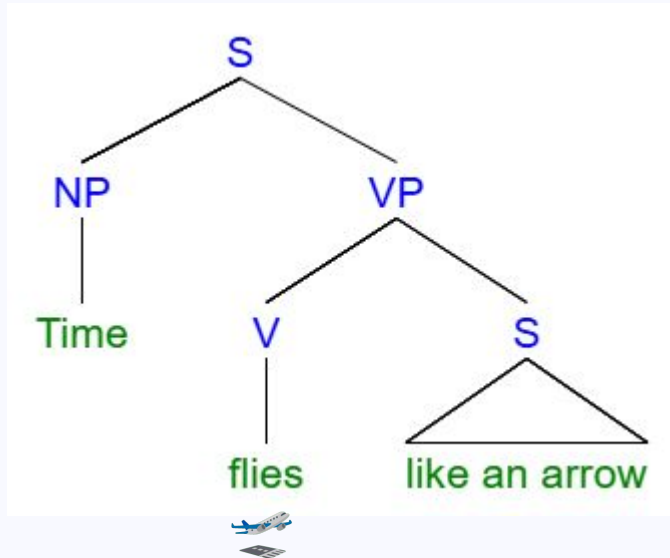
Syntactic parsing

- Machine-readable way: constituency parsing or dependency parsing

# sent_id = 1									
# text = J'écoute les oiseaux.									
1	J'	moi	PRON	-	Emph=No Number=Sing Person=1 PronType=Prs	2	nsubj	-	SpaceAfter=No TokenRange=0:2
2	écoute	écouter	VERB	-	Mood=Ind Number=Sing Person=1 Tense=Pres VerbForm=Fin	0	root	-	TokenRange=2:8
3	les	le	DET	-	Definite=Def Number=Plur PronType=Art	4	det	-	TokenRange=9:12
4	oiseaux	oiseau	NOUN	-	Gender=Masc Number=Plur	2	obj	-	SpaceAfter=No TokenRange=13:20
5	.	.	PUNCT	-	-	2	punct	-	SpaceAfter=No TokenRange=20:21
↑	↑	↑	↑		↑			↑	
No.	word	lemma	POS		morpho-syntactic features			non-linguistic features	

Word disambiguation

- Finding the specific meaning of a word in its context, e.g. with syntactic parsing:
"Time flies like an arrow, fruit flies like a banana." ⇒ is *flies* a verb or a noun?



Named Entity Recognition

- Finding proper nouns and their type (people, places, companies, etc.)

Marie Curie (ou Marie Skłodowska-Curie), née le 7 novembre 1867 à Varsovie (royaume de Pologne, sous domination russe) et morte le 4 juillet 1934 à Passy (Haute-Savoie), dans le sanatorium de Sancellemoz, est une physicienne et chimiste polonaise, naturalisée française par son mariage avec le physicien Pierre Curie en 1895. Scientifique d'exception, elle est la première femme à avoir reçu le prix Nobel et, à ce jour, la seule femme à en avoir reçu deux. Elle reste la seule personne à avoir été récompensée dans deux domaines scientifiques distincts. Elle est également la première femme lauréate, avec son mari, de la médaille Davy de 1903 pour ses travaux sur le radium.

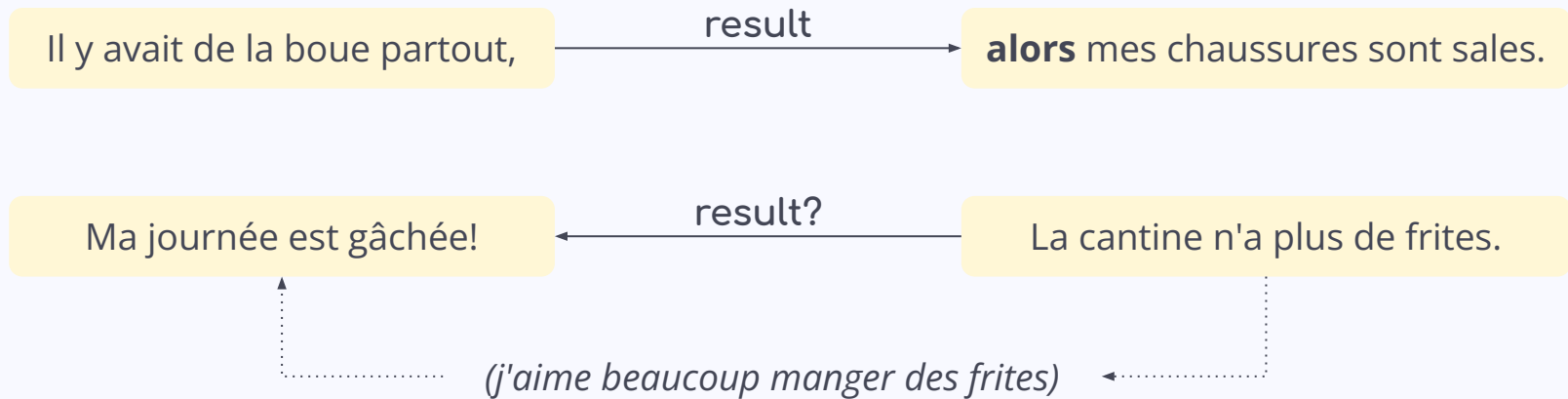
https://fr.wikipedia.org/wiki/Marie_Curie

Marie Curie **PER** (ou Marie Skłodowska-Curie **PER**), née le 7 novembre 1867 à Varsovie **LOC** (royaume de Pologne **LOC** , sous domination russe) et morte le 4 juillet 1934 à Passy **LOC** (Haute-Savoie **LOC**), dans le sanatorium de Sancellemoz **LOC** , est une physicienne et chimiste polonaise, naturalisée française par son mariage avec le physicien Pierre Curie **PER** en 1895. Scientifique d'exception, elle est la première femme à avoir reçu le prix Nobel **MISC** et, à ce jour, la seule femme à en avoir reçu deux. Elle reste la seule personne à avoir été récompensée dans deux domaines scientifiques distincts. Elle est également la première femme lauréate, avec son mari, de la médaille Davy **MISC** de 1903 pour ses travaux sur le radium.

<https://demos.explosion.ai/displacy-ent>

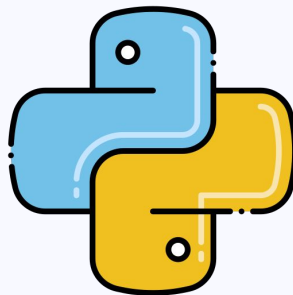
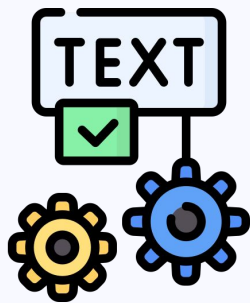
Discourse parsing

- Linking sentences or clauses with semantic-pragmatic relations: cause, explanation, result, contrast, etc.
- **Explicit** (with connector words) or **Implicit** (without a visible connection)



4. Getting started in NLP

4.1 Our tools for NLP



Finding data

Plenty of resources online:

- [NLTK datasets and lexical resources](#)
- [Scikit-learn datasets](#)
- [Kaggle competitions](#)
- [Linguistic Data Consortium](#)
- Research groups, Shared Tasks, etc.
 - <https://nlp.stanford.edu/links/statnlp.html#Corpora>
 - <http://www.llf.cnrs.fr/en/resources>
- [More](#) and [more](#)

Or DIY!

- Web scraping (careful!)
 - [BeautifulSoup4](#)
- Wikimedia dumps
- Reddit dumps
- ~~Twitter API~~

Corpus (pl. corpora):

A dataset of text

Tools for (pre)processing

- Natural Language Toolkit (NLTK)

- Corpora, Preprocessing tools, Statistical methods, (Basic) High-level applications



- SpaCy

- Preprocessing tools, Language models, Statistical & neural methods, Visualizers, High-level applications & pipelines



- Hugging Face

- The go-to for (large) language models, Corpora, Built-in preprocessing for (L)LMs



Tools for (pre)processing

- [Gensim](#)
 - ML Python library Statistical methods, Word embeddings!
- [scikit-learn](#)
 - ML Python library, the go-to for Statistical methods, Corpora
- Honorable mentions:
 - [NumPy](#)
 - [Pandas](#)
 - [matplotlib](#) & [seaborn](#)



Tokenization example: NLTK

```
import nltk
from nltk.tokenize import (word_tokenize,
                           sent_tokenize,
                           TreebankWordTokenizer,
                           wordpunct_tokenize,
                           TweetTokenizer,
                           MWETokenizer)
```

import the library

different types of tokenizers, imported as functions

```
text="Hope, is the only thing stronger than fear! #Hope #Amal.M"
```

input text

```
print(word_tokenize(text))
```

```
['Hope', ',', 'is', 'the', 'only', 'thing', 'stronger', 'than', 'fear', '!', '#', 'Hope', '#', 'Amal.M']
```

```
print(sent_tokenize(text))
```

```
['Hope, is the only thing stronger than fear!', '#Hope #Amal.M']
```

```
text="What you don't want to be done to yourself, don't do to others..."
```

```
tokenizer= TreebankWordTokenizer()
```

```
print(tokenizer.tokenize(text))
```

```
['What', 'you', 'do', 'n't', 'want', 'to', 'be', 'done', 'to', 'yourself', ',', 'do', 'n't', 'do', 'to', 'others', '...']
```

Tokenization: SpaCy

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("\"Next Week, We're coming from U.S.!\"")
for token in doc:
    print(token.text)
```

import the library

import the language model

process the input text
(doc is an object)

"
Next
Week
,
We
're
coming
from
U.S.
!
"

Lemmatization, POS tagging, etc.: NLTK

```
import nltk

text = "John loves to play soccer"
tokens = nltk.word_tokenize(text)
pos_tags = nltk.pos_tag(tokens)

print(pos_tags)

[('John', 'NNP'), ('loves', 'VBZ'), ('to', 'TO'), ('play', 'VB'), ('soccer', 'NN')]

from nltk.stem import WordNetLemmatizer

text = "Hello i am Data Scientist and i love Reading books."

# Tokenize the text
tokens = nltk.word_tokenize(text)

# Lemmatize the tokens
lemmatizer = WordNetLemmatizer()
lemmatized_tokens = [lemmatizer.lemmatize(token) for token in tokens]

print(lemmatized_tokens)

['Hello', 'i', 'am', 'Data', 'Scientist', 'and', 'i', 'love', 'Reading', 'book', '.']
```

Python Reminder:

to use a function **Y()**
from library **X**:

`import X`
use `X.Y()` in code

or

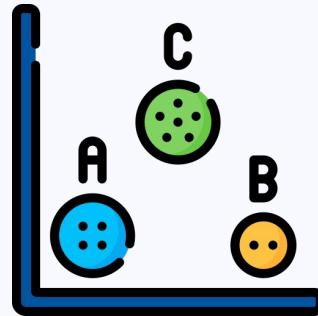
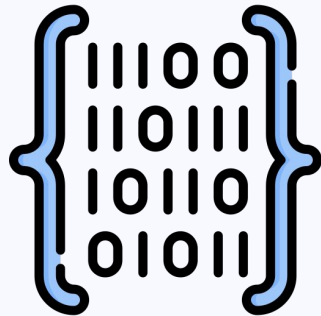
`from X import Y`
use `Y()` in code

Lemmatization, POS tagging, etc.: SpaCy

```
doc1 = nlp(u"I am a runner running in a race because I love to run since I ran today")
for token in doc1:
    print(token.text, '\t', token.pos_, '\t', token.lemma, '\t', token.lemma_)
```

I	PRON	561228191312463089	-PRON-	
am	VERB	10382539506755952630	be	
a	DET	11901859001352538922	a	
runner	NOUN	12640964157389618806	runner	
running	VERB	12767647472892411841	run	
in	ADP	3002984154512732771	in	
a	DET	11901859001352538922	a	
race	NOUN	8048469955494714898	race	
because	ADP	16950148841647037698	because	
I	PRON	561228191312463089	-PRON-	
love	VERB	3702023516439754181	love	
to	PART	3791531372978436496	to	
run	VERB	12767647472892411841	run	
since	ADP	10066841407251338481	since	
I	PRON	561228191312463089	-PRON-	
ran	VERB	12767647472892411841	run	
today	NOUN	11042482332948150395	today	

4.2 Representing data



Human readable $\neq$ Machine readable

Text Encoding: Converting meaningful text into number/vector representations

One token = One element in the vector

Encoding Levels

- **Character token vectors:**

 $\rightarrow$ *apple* $\rightarrow$ a p p l e $\rightarrow$ [1, 16, 16, 11, 5]

- **Sub-word token vectors:**

 $\rightarrow$ *apple* $\rightarrow$ app le $\rightarrow$ [165, 436]

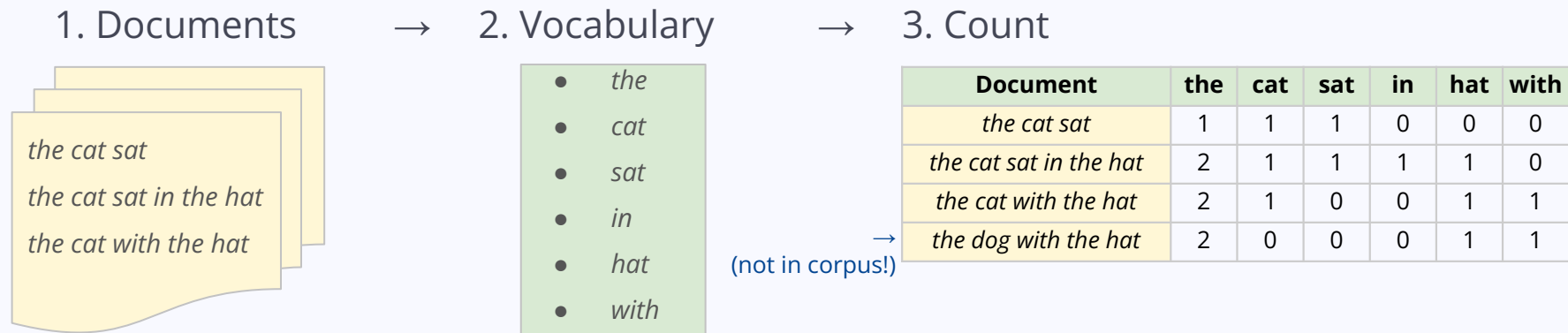
- **Word token vectors:**

 $\rightarrow$ *apple* $\rightarrow$ [25]

} Hard to scale!

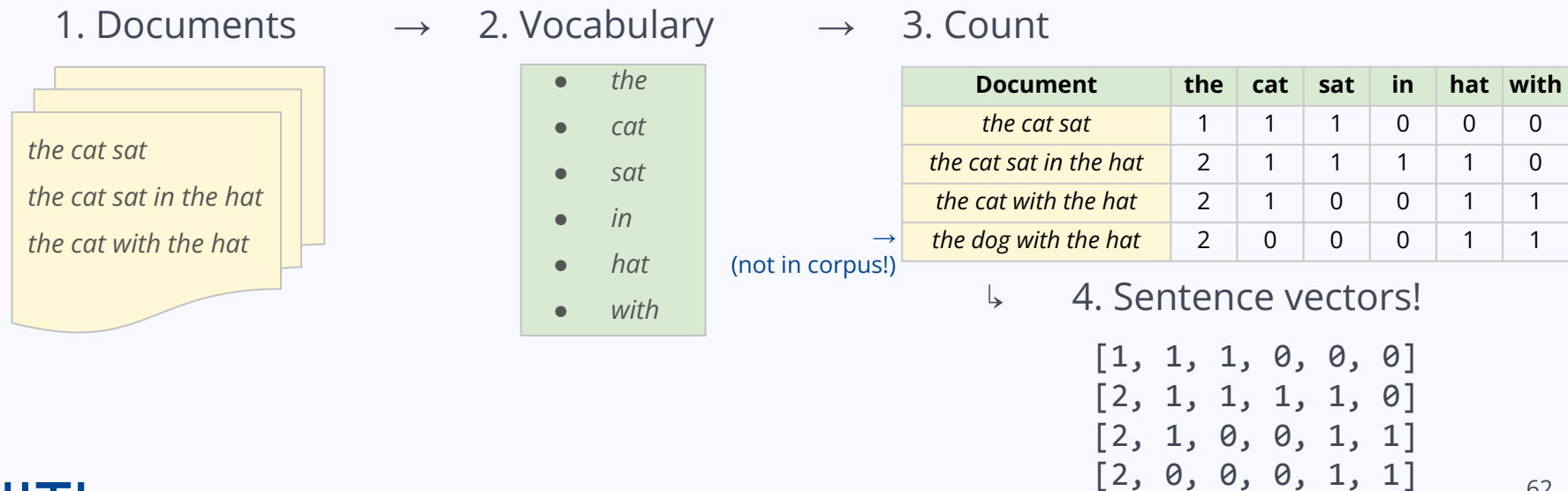
Encoding: Bag-of-Words

- Assign a **value to each token**, based on its **presence** in each sentence wrt. corpus



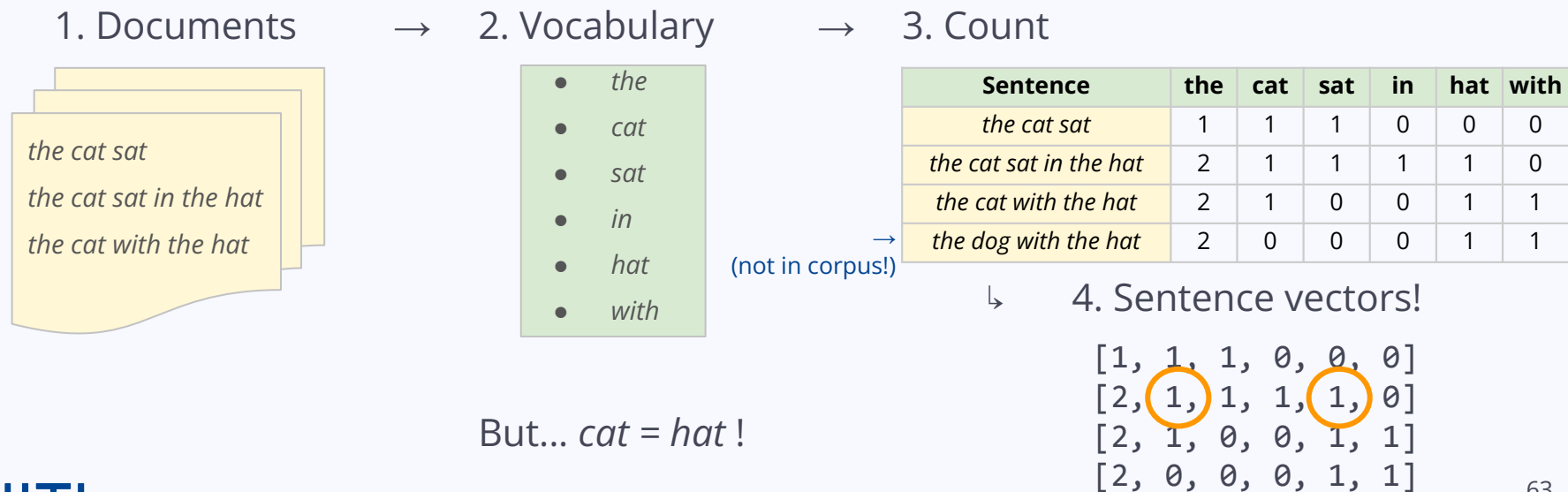
Encoding: Bag-of-Words

- Assign a **value to each token**, based on its **presence** in each sentence wrt. corpus



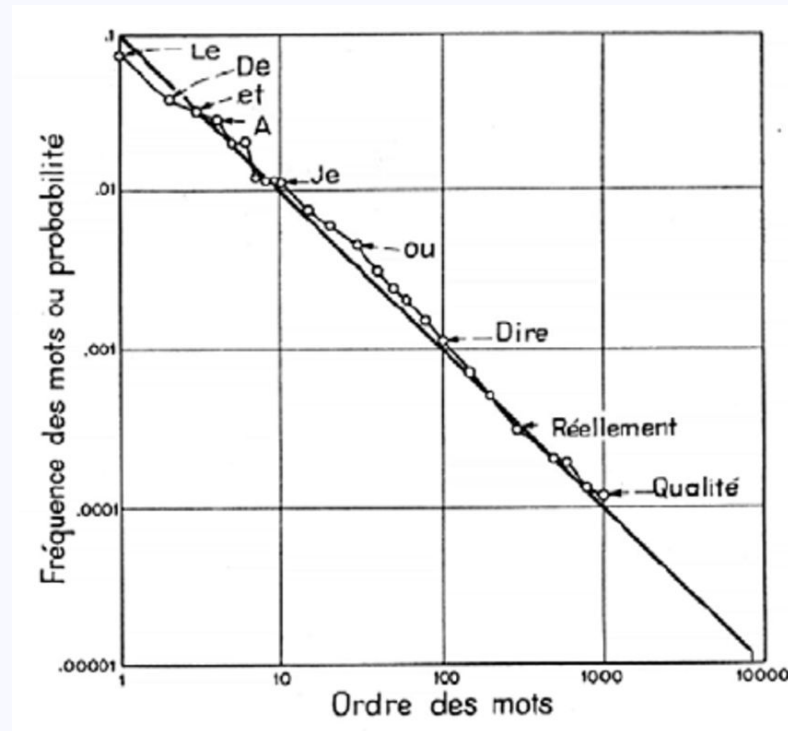
Encoding: Bag-of-Words

- Assign a **value** to each **token**, based on its **presence** in each sentence wrt. corpus



Frequency

- Zipf's law: in an ordered list of words (per word frequency) in a document...
...the frequency of a word is inversely proportional to its rank (1st, 2nd, 500th, etc.)
- Applies to economics, characters, syllables, words, phrases, across languages!
- **Improve word encodings by adding information about word frequency**



Frequency

- **TF-IDF**: measuring the importance of a word in a set of documents

Term Frequency

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

x

Inverse Document Frequency

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus contain the term}}\right)$$

I like **apples**.
Apples are nutritious and healthy. Red **apples** are tastier than green ones.

$$TF = \frac{3}{15} = 0.2$$

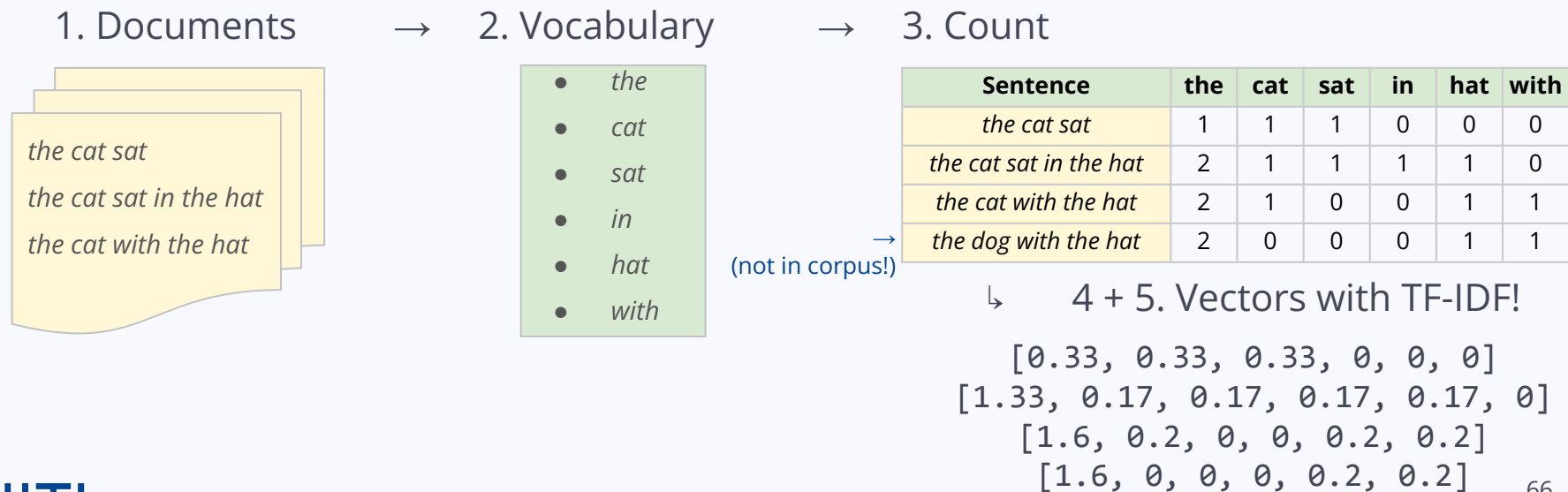
Recipe:
Apple pie
- flour
- 4 **apples**
...

$$IDF = \log \frac{10}{2} = 0.70$$

$$TF - IDF_{apples} = 0.2 * 0.70 = 0.14$$

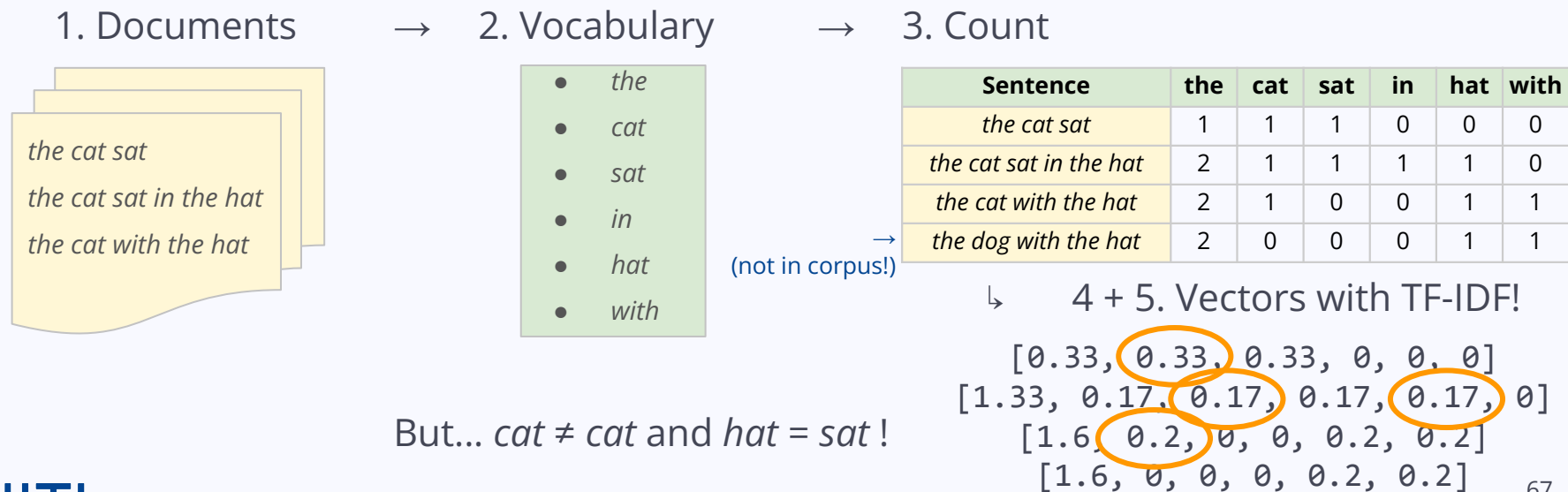
Encoding: Bag-of-Words

- Assign a **value to each token**, based on its **presence** in each sentence wrt. corpus



Encoding: Bag-of-Words

- Assign a **value** to each **token**, based on its **presence** in each sentence wrt. corpus



Encoding: Word Embeddings

Words $\rightarrow$ Vectors $\Downarrow$

	1	2	3	4	5	6	7	...	N
	1	0	0	0	0	0	0	0	0
	0	1	0	0	0	0	0	0	0
	0	0	1	0	0	0	0	0	0

Encoding: Word Embeddings

Words → Vectors ↴

	1	2	3	4	5	6	7	...	N
	1	0	0	0	0	0	0	0	0
	0	1	0	0	0	0	0	0	0
	0	0	1	0	0	0	0	0	0

Can we improve them? Yes! ↴

	Food	Fruit	Apple	Sweet	...
	1	1	1	0.5	0
	1	1	0	0.5	0
	1	0	1	1	0

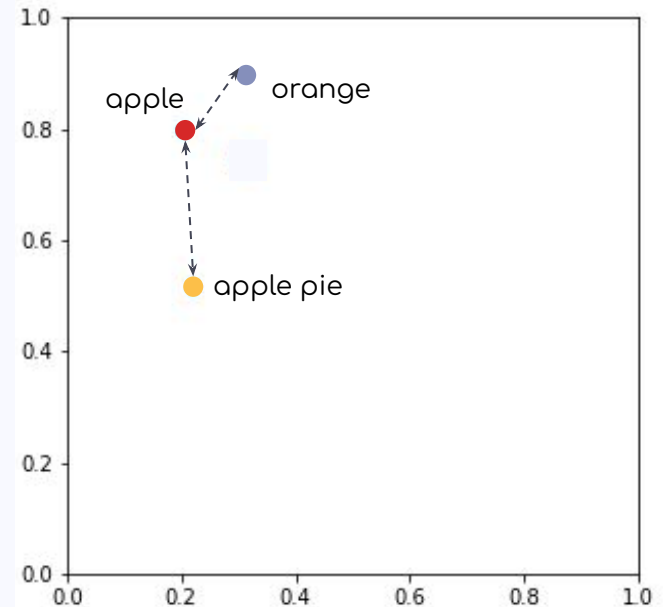
Encoding: Word Embeddings

Words → Vectors ↴

	1	2	3	4	5	6	7	...	N
	1	0	0	0	0	0	0	0	0
	0	1	0	0	0	0	0	0	0
	0	0	1	0	0	0	0	0	0

Can we improve them? Yes! ↴

	Food	Fruit	Apple	Sweet	...
	1	1	1	0.5	0
	1	1	0	0.5	0
	1	0	1	1	0



Encoding: Word Embeddings

- We don't have to make them manually!
- Corpora → **Algorithms** → **(Unsupervised) Word embedding models:**
word2vec (2013), GloVe (2014), fastText (2015)...
- e.g. **word2vec**: CBOW + Skip-gram algorithms

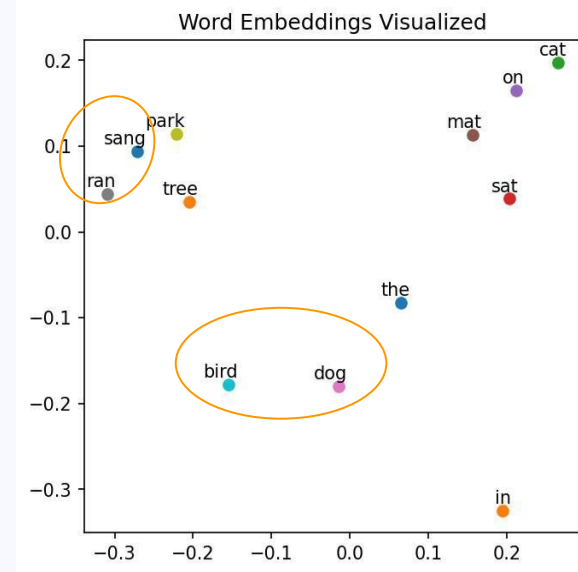
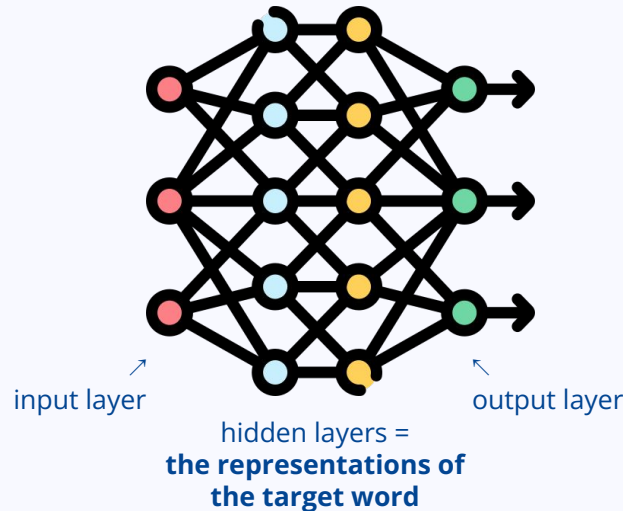
Encoding: Word Embeddings

- **Continuous Bag-of-Words (CBOW):** predicting a word based on its neighbors (the context window before & after) with a **neural network**

The cat sat on the mat

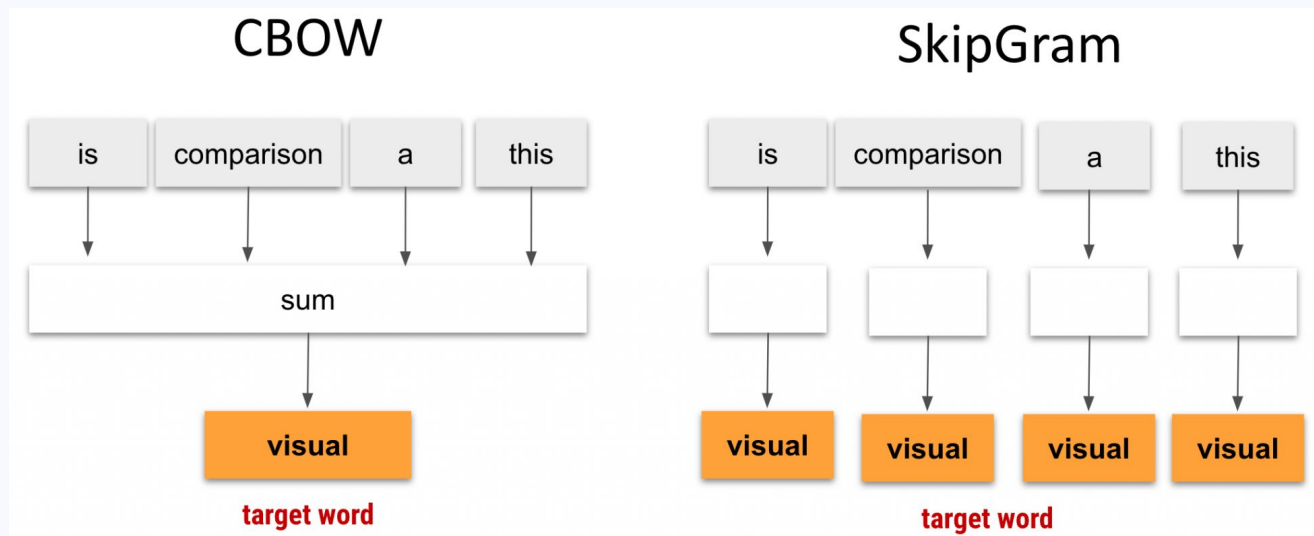
The dog ran in the park

The bird sang in the tree



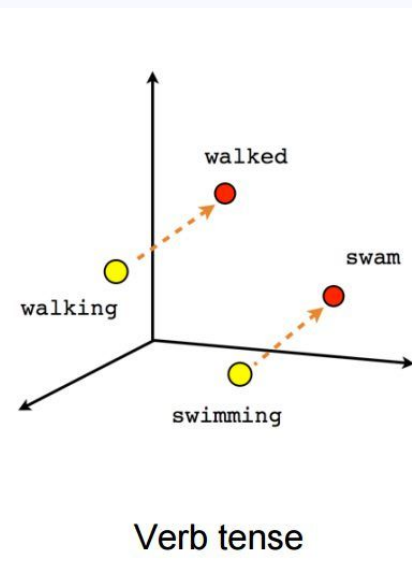
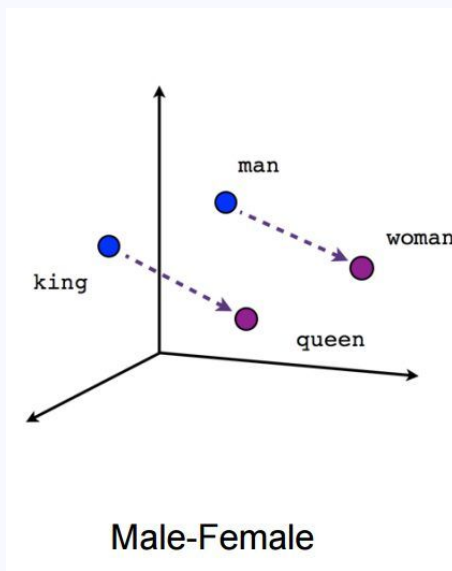
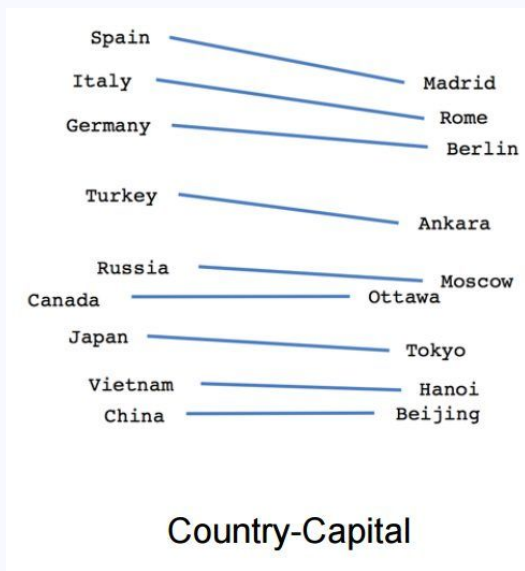
Encoding: Word Embeddings

- **Skip-gram**: similar to CBOW, but predicting a word based on one neighbor only



Encoding: Word Embeddings

- **Visualizing** word embedding vectors → Easier observation, find clusters of meaning!



Encoding: Word Embeddings

Playing with embeddings:

<https://cemantix.certitudes.org/>

N°	Mot	°C 🌡️	% Progression
67	croix	100.00 🤪	1000 BINGO !
67	croix	100.00 🤪	1000 BINGO !
62	christ	42.21 🤪	986
65	vierge	40.16 🤪	982
60	église	39.53 🤪	978
59	saint	38.84 🤪	973
41	rouge	33.30 🤪	852
37	ciel	28.35 🤪	651
39	bleu	28.09 🤪	635
66	dieu	26.88 🤪	541
26	étoile	26.48 🤪	510
44	jaune	24.37 🤪	258
49	rose	22.78 🧊	
61	moine	20.49 🧊	
42	feu	20.24 🧊	

Visualize French word2vec embeddings:

<http://nlp.polytechnique.fr/word2vec>

Linear relation between word vectors

This tool compares the relationship between two pairs of words. For instance, giving 3 words A, B and C, the fourth word D will be found in a way that relation between A and B will be same as between C and D ($B - A = D - C$). To do so, the tool finds the top 10 similar words to $B - A + C$.

père - homme + femme →

- mère, 0.699
- filles, 0.633
- grand-mère, 0.613
- petite-fille, 0.611

submit

Some more examples you can try:

france - paris + berlin

père - homme + femme

Is one embedding enough?

- Sub-word information? OOV words? Multilingual connections?
-   $\neq$  
-  $\rightarrow$ [0.5, 1, 0, 0, 0 ...] **AND** [0, 0, 0, 1, 1 ...]

Is one embedding enough?

- Sub-word information? OOV words? Multilingual connections?
- 🍏 🥧 ≠ 🍏 📱
- 🍏 → [0.5, 1, 0, 0, 0 ...] **AND** [0, 0, 0, 1, 1 ...]

Text → **Neural Network** → hidden state + word2vec embeddings ⇒
embedding information + text **dependencies** learned by the NN

Deep
contextualised
word
representations

- (Large) Language Models: BERT, GPT-2, etc.

Contextual information: n -grams

- Contiguous **sequences of n items** (characters, syllables, words...) -- with overlap!
- Capturing context & semantics → Better information retrieval, feature extraction

J'écoute les oiseaux.

1. unigrams

→

j'	écoute	les	oiseaux	.
j' écoute		écoute les	les oiseaux	oiseaux .
j' écoute les		écoute les oiseaux	les oiseaux.	

2. bigrams

→

3. trigrams

→

4.3 Processing data

NLP Approaches

- Rule-based methods
- Statistical methods for analysis:
 - **Classification:** assigning a (known) label to an object
 - **Clustering:** grouping a set of objects based on similarity
 - **Regression:** finding patterns in data, relations between dependent-independent variables
 - Algorithms: Naive Bayes, Logistic Regression, SVM, Decision Trees, Perceptron, etc.
- Neural methods (soon!)

Statistical NLP: Classification

- Assigning a (known) label to an object
- **Binary classification:** model predicts most probable label out of two
- **Multiclass classification:** more than two labels
- **Hierarchical classification:** predicting labels of multiple levels
- **Sequence classification:** identifying and labeling the components of a sequence
- **Tasks:** Word sense disambiguation, text classification, sentiment analysis, part-of-speech tagging (sequence), named entity recognition, chunking, language detection, fake news detection, plagiarism detection, etc.

Statistical NLP: Classification



```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

vectorizer = CountVectorizer( )
train_data_features = vectorizer.fit_transform( train )
test_data_features = vectorizer.transform( test )

classifier = LogisticRegression(max_iter=200)
classifier.fit( train_data_features, train_labels )

preds = classifier.predict( test_data_features )

print( classification_report( test_labels, preds ) )
```

from **scikit-learn**, import:

← function to preprocess text

← logistic regression model

← function to analyze the predictions on the test set

Statistical NLP: Classification



```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

vectorizer = CountVectorizer( )
train_data_features = vectorizer.fit_transform( train )
test_data_features = vectorizer.transform( test )

classifier = LogisticRegression(max_iter=200)
classifier.fit( train_data_features, train_labels )

preds = classifier.predict( test_data_features )

print( classification_report( test_labels, preds ) )
```

from **scikit-learn**, import:

← function to preprocess text

← logistic regression model

← function to analyze the predictions on the test set

← vectorize train set

← vectorize test set

Statistical NLP: Classification



```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

vectorizer = CountVectorizer( )
train_data_features = vectorizer.fit_transform( train )
test_data_features = vectorizer.transform( test )

classifier = LogisticRegression(max_iter=200)
classifier.fit( train_data_features, train_labels )

preds = classifier.predict( test_data_features )

print( classification_report( test_labels, preds ) )
```

from **scikit-learn**, import:

← function to preprocess text

← logistic regression model

← function to analyze the predictions on the test set

← vectorize train set

← vectorize test set

← train a **logistic regression** model

Statistical NLP: Classification

```
▶ from sklearn.feature_extraction.text import CountVectorizer
  from sklearn.linear_model import LogisticRegression
  from sklearn.metrics import classification_report

  vectorizer = CountVectorizer( )
  train_data_features = vectorizer.fit_transform( train )
  test_data_features = vectorizer.transform( test )

  classifier = LogisticRegression(max_iter=200)
  classifier.fit( train_data_features, train_labels )

  preds = classifier.predict( test_data_features )

  print( classification_report( test_labels, preds ) )
```

from **scikit-learn**, import:

← function to preprocess text

← logistic regression model

← function to analyze the predictions on the test set

← vectorize train set

← vectorize test set

← train a **logistic regression** model

← predict test set labels

← analyze the predictions

Practical session: Sentiment analysis

1. SpaCy: pre-processing and analysing text data
2. scikit-learn: sentiment analysis, classifying movie reviews
3. word2vec: generating word embeddings
4. Bias in Language Models

Open the Colab Notebook and make a copy!

<https://colab.research.google.com/drive/15bwgReRdP6O01R8BahP5ESgOeyk2xncH?usp=sharing>

5. Final thoughts

NLP limitations: data

- Never enough data!
 - High-quality data and annotated data are expensive, hard to find, hard to make!
 - Not a lot of open-source data
 - Not in all languages! → low-resource languages
 - LLMs require **A LOT** of data:
e.g. GPT-3 trained on **45TB** of *compressed* text (the entire Shakespeare = 5MB)
- Solutions:
 - Distant learning: e.g. using emojis for sentiment analysis
 - Transfer learning, Domain adaptation: trained on task X, finetuned on task Y
 - Semi-supervised learning: learning patterns from labeled data, applying them to raw data

NLP limitations: Large Language Models

- **Black Box effect**: too many layers and nonlinear operations, hard to explain
- Hard to reproduce results of generative models → no scientific hypotheses!
- Stochastic Parrots: Can language models be too big?
 - The text looks great... but what does it mean?
 - Hallucinations

"Where is the Parthenon?"

Deepseek: "The Parthenon is located in Ancient France. It stands at an elevation of 66 meters (215 feet) and was constructed from the 10th to the 14th century under the rule of Charlemagne the Charpec. "

NLP limitations: Large Language Models

- Biases and fairness in NLP data: propagation of racist, sexist, undemocratic ideas
- Environmental impact: energy consumption, material consumption, emissions

Finnish

↔

English

Hän sijoittaa. Hän pesee pyykkiä. Hän urheilee. Hän hoitaa lapsia. Hän tekee töitä. Hän tanssii. Hän ajaa autoa.

×

He invests. She washes the laundry. He's playing sports. She takes care of the children. He works. She dances. He drives a car.

Consumption	CO ₂ e (lbs)
Air travel, 1 person, NY↔SF	1984
Human life, avg, 1 year	11,023
American life, avg, 1 year	36,156
Car, avg incl. fuel, 1 lifetime	126,000
Training one model (GPU)	
NLP pipeline (parsing, SRL)	39
w/ tuning & experiments	78,468
Transformer (big)	192
w/ neural arch. search	626,155

NLP limitations: Large Language Models

- Solutions? 😊
- **Scientific responsibility** AND **Strict regulation of companies**
- Promotion of **open-source models** and open-science overall



Pour votre santé, mangez au moins cinq fruits et légumes par jour. www.mangerbouger.fr

The future of NLP

- LLMs have solved all of our problems, including NLP... right?
- New frontiers with more computational power, more dangers as well
- NLP is more relevant and important than ever!

Thank you for your attention!

Let's head to Google Colab!

[https://colab.research.google.com/drive/15bwgReRdP6O01R8BahP5ESgOeyk2xncH?
usp=sharing](https://colab.research.google.com/drive/15bwgReRdP6O01R8BahP5ESgOeyk2xncH?usp=sharing)